

Manipulating & validating data

Raw data is almost never ready to use. Before it can tell a story, you have to manipulate it into shape, validate that it's clean, and test that everything you built actually works. These are the engine-room skills of data analysis.

What this key knowledge covers

This summary distils everything you need for **KK12 — Manipulating & validating data** in Data Analysis. Work through the explanations, then use the worked good-vs-weak answers to see exactly how marks are won and lost. Tick off the revision checklist at the end before a SAC or exam.

Study summary

Where this fits

KK12 is the "make it work" dot point — the hands-on middle of the Data Analysis outcome.

You've already learned what data *is* (KK01–KK04), how to judge its *quality* (KK03), and how databases and spreadsheets are *structured* (KK10–KK11). KK12 is where you finally **do things to the data**: reshape it, guard it against errors, and prove your work is correct. KK13 then narrows in on the specific statistics — average, median, count, standard deviation — so keep this dot point about the broad *techniques*, not those particular calculations.

*"Functions and techniques to **manipulate**, **validate** and **test** data, including formulas and functions, charts and graphs, and Structured Query Language (SQL) queries."*

Because the wording uses including, all three tool-sets — formulas/functions, charts/graphs and SQL queries — are directly examinable.

Meet the running scenario: SnackTrack

Mia is a Year 11 student who runs the data behind her school canteen's **SnackTrack** loyalty app. Every Friday the till exports a messy file of lunch orders. Across this module, Mia will manipulate that file into insight, validate it so next week's orders stay clean, and test that her formulas, queries and charts are right. Same dataset, three jobs.

Three jobs, one dataset

Manipulate, validate, test — they sound similar but answer three different questions.

□ Manipulate

"Reshape it." Change the data's form or pull out a subset so it answers your question — calculate, sort, filter, query, chart.



☐ **Validate**

"**Is it reasonable?**" Check incoming data is sensible *before* it's accepted — existence, type and range checks stop garbage getting in.

✓ **Test**

"**Did it work?**" Prove your solution behaves correctly using a test table of expected vs actual results, and verify charts match their data.

The kitchen analogy

Manipulating is the *cooking* — chopping, mixing and plating raw ingredients into a dish.

Validating is the *quality check at the door* — rejecting a delivery of off milk before it ever reaches the kitchen. **Testing** is *tasting before you serve* — confirming the dish is actually what the recipe promised. Skip any one and a customer gets a nasty surprise.

Validation happens at the *point of entry* and asks "is this value *reasonable?*" — it can never guarantee a value is *true*. A perfectly reasonable quantity of 3 still passes validation even if the student actually bought 5. **Testing** happens *after* you build, and asks "does my solution produce the *expected* result?"

Lab A • The data pipeline

Watch a raw order travel through all three stations. Click a machine to see its job.

Manipulating data

Three tool-sets do the heavy lifting — and the study design names all three.

To *manipulate* data means to change its form, organisation or content so it answers a question. In VCE Applied Computing you do this with three software tool-sets, each suited to a different job:

f Formulas & functions

Inside a **spreadsheet**. Calculate new values, combine text, look things up, sort and filter rows.

☐ **SQL queries**

Inside a **relational database**. Select exactly the rows and columns you need from large tables — fast.

▣ **Charts & graphs**

In a **spreadsheet or visualisation tool**. Turn rows of numbers into a shape the eye can read instantly.

A real workflow usually chains them: **SQL** pulls the right slice out of a big database → you **import** it into a spreadsheet → **formulas** refine and calculate → a **chart** presents the finding. Mia uses exactly this path below.

Formulas & functions

Mia's raw export, refined one column at a time.

A **formula** is any expression you type into a cell that starts with =. A **function** is a named, built-in operation (like SUM or IF) you use inside a formula. Here is Mia's raw SnackTrack export — the till gave her quantity and unit price, but not the line totals she needs.

The four manipulation moves you'll be expected to recognise:

① Calculate

Derive new values: =B2*C2 for a line total, =SUM(D2:D5) for a grand total, =D2/\$D\$6 for each item's share.

② Combine / look up

=CONCAT(A2," x",B2) builds a label; =VLOOKUP(A2,menu,3,0) fetches a price from the menu table.

③ Decide with logic

=IF(D2>10,"Big order","Small") tags rows by a rule — the spreadsheet equivalent of an *if* statement.

④ Sort & filter

Reorder rows by line total (descending) to surface top sellers; filter to *Category = Drink* to focus a question.

When Mia copies =D2/\$D\$6 down the column, D2 shifts to D3, D4 ... (relative) but \$D\$6 stays locked on the total (absolute). Mixing these up is the single most common spreadsheet bug — and a favourite exam target.

SQL queries

When the data lives in a database, you ask for what you want in Structured Query Language.

A spreadsheet is fine for a few hundred rows, but a term's worth of orders lives in a **relational database**. **SQL** (Structured Query Language) lets Mia request *exactly* the rows and columns she needs without scrolling. Every query you'll meet at this level is built from four clauses:

Try it on Mia's orders table. Change the clauses and watch both the query *and* the result update live:

Say it as a sentence: "*Select these columns from this table where this condition is true, ordered by this column.*" If you can narrate a query in plain English, you can usually write one. Watch out for the string-vs-number rule: text values need quotes (WHERE category = 'Drink') but numbers do not (WHERE qty > 3).

Charts & graphs

A chart is a manipulation too — it turns a column of numbers into a shape you read in a second.

Once the totals are calculated, Mia builds a chart so the canteen manager can see top sellers at a glance. Choosing the *right* chart matters: a **bar/column chart** compares categories, a **line chart** shows change over time, and a **pie chart** shows parts of a whole. Toggle the data below and watch the bars rebuild in 3D:

Validating data

Three checks decide whether a value is reasonable enough to let in.

Validation is the process of checking that data entered into a solution is *reasonable, complete and sensible* — done *at the point of entry*, before the value is stored. It can't prove a value is *true*; it only blocks the obviously wrong. The three techniques you must know by name:

∃ Existence check

Is there a value at all? A required field can't be left blank. "*Item name must not be empty.*"

Type check

Is it the right kind of data? Quantity must be a *number*, not text. "*five is rejected; 5 is accepted.*"

□ Range check

Is the value within sensible limits? "*Quantity must be between 1 and 50.*" Blocks -2 and 999.

Validation only catches the *unreasonable*. If Mia types 3 sausage rolls when she really sold 5, every check passes — the value is reasonable, just wrong. Catching *that* kind of error needs **verification** (e.g. double-entry or proofreading against the source), which is a different idea.

In a **spreadsheet** you set it with *Data Validation* rules (whole number, between 1 and 50). In a **database** you set *field properties* and validation rules on the table, so bad records are rejected on import. Same three checks, two homes.

Lab B • The validation gates

Build an order record, then send it through three gates. Will it survive?

Set the three properties of an incoming order, then press **send**. The record (a glowing cube) flies toward three gates — Existence, Type, Range. A green gate lets it pass; a red gate bounces it back. Make it through all three and the record is accepted into the database.

Testing data

A formula that *runs* isn't the same as a formula that's *right*. Prove it with a test table.

Once Mia has built her formulas, queries and chart, she has to **test** them. The standard tool is a **test table** with four columns. You choose *deliberate* test data — including tricky edge cases — work out the expected result by hand, run it, and compare.

T3's mismatch is the whole point of testing: it exposed that Mia's range rule was set to <1000 instead of <=50. Without the test, that bug ships.

Choose test data on purpose

Cover **valid** (normal), **invalid** (should be rejected) and **boundary** (right on the limit — 0, 1, 50, 51) cases. Boundaries catch the most bugs.

Expected before actual

Work out the **expected** result *by hand first*. If you fill it in after running the solution, you're just copying the output, not testing it.

Validation guards the *input* at entry time. **Testing** checks the *whole solution* after you build it. A range check is validation; confirming that range check actually fires is testing.

Lab C · Verify the chart

Testing isn't only for formulas — a finished visualisation must match its data too.

Mia's chart software re-typed one value by mistake, so a bar no longer matches the spreadsheet. **Verifying** a visualisation means checking every bar against its source value — exactly the kind of question VCAA loves. The data table is correct. **Click the bar that doesn't match its number.**

Exam traps

The mix-ups that quietly cost marks. Learn the name, dodge the trap.

P·E·L writing trainer

Structure every short-answer like an examiner: **Point** · **Example** · **Link**. Pick a concept to see it modelled.

Markers award the **Point** (the definition/term), the **Example** (it proves you can apply it — usually worth its own mark) and the **Link** (ties it back to the scenario, which separates a C from an A). Most students give only the Point and cap themselves at half marks.

Exam practice

Five VCAA-style questions · 20 marks. Read each, attempt it, then reveal a weak and strong response and self-mark.

Distinguish between *validating* data and *testing* a solution.

"Validating and testing both check that the data is correct so there are no mistakes."

Examiner: treats the two as the same thing and uses "correct", which validation can't guarantee. No distinction made → 0–1.

"Validation checks that data is *reasonable and sensible at the point of entry* (e.g. a range check rejecting a quantity of 999). Testing checks that the *completed solution behaves as expected* after it's built, using a test table of expected vs actual results."

Examiner: clear contrast on timing and purpose, each with an example → full marks.

- 1 mark — validation = checking data is reasonable/sensible at entry
- 1 mark — testing = checking the built solution produces expected results

Name and describe **three** validation techniques. For each, give an example based on a canteen order quantity.

Cheat sheet

The whole dot point on one screen.

Common traps & misconceptions

Exam trap

T1 "Validation" used to mean "make it correct" Validation only checks data is *reasonable*, never *true*. Fix: say validation "checks data is sensible at entry"; use *verification* for checking against the real source.

Exam trap

T2 Naming the wrong three checks The trio is *existence*, *type*, *range*. Students invent "format check" or "length check" (real, but not the named three). Fix: memorise existence · type · range.

Exam trap

T3 Confusing manipulate / validate / test They answer different questions: reshape it · is it reasonable · did it work. Fix: tie each to its question before you write.

Exam trap

T4 SQL string quotes dropped Text values in WHERE need single quotes (= 'Drink'); numbers don't. Fix: quote text, leave numbers bare.

Exam trap

T5 Filling "Expected" after running Copying the output into the expected column proves nothing. Fix: always calculate expected by hand *first*.

Exam trap

T6 Forgetting boundary test data "Test it works" with one normal value misses the bugs. Fix: include valid, invalid *and* boundary cases (0, 1, max, max+1).

How to answer exam questions

Read the **command word** first — it sets how much to write. *State/Identify* wants a word or phrase; *Describe* wants features; *Explain* wants reasons and links; *Justify/Evaluate* wants a judgement backed by evidence. Always pull in the **scenario detail** rather than answering in the abstract. The examples below contrast a weak response with a strong one for the same question.

Q1. Practice question · 1 mark

(2) — validation = checking data is reasonable/sensible at entry — testing = checking the built solution produces expected results

✗ A weak answer

er response "Validating and testing both check that the data is correct so there are no mistakes." Examiner: treats the two as the same thing and uses "correct", which validation can't guarantee. No distinction made 0-1.

✓ A strong answer

er response "Validation checks that data is *reasonable and sensible at the point of entry* (e.g. a range check rejecting a quantity of 999). Testing checks that the *completed solution behaves as expected* after it's built, using a test table of expected vs actual results." Examiner: clear contrast on timing and purpose, each with an example full marks.

Q2. Practice question · 1 mark

(3) each for existence, type and range — name + correct description/example No mark for format/length/presence checks unless tied to the three named techniques

✗ A weak answer

er response "You can use a range check, a format check and a length check to make sure the quantity is okay." Examiner: range check is correct, but format and length aren't the named trio for this course, and there are no examples. 1.

✓ A strong answer

er response "**Existence check** — the quantity field can't be left blank. **Type check** — the quantity must be a number, so five is rejected but 5 is accepted. **Range check** — the quantity must be between 1 and 50, so 999 is rejected." Examiner: correct trio, each with a quantity example. 3.

Q3. Practice question

(4) (a) 1 — correct SELECT columns + FROM + WHERE qty > 3; 1 — correct ORDER BY qty DESC
(b) 1 — names a genuine advantage (speed / accuracy / only relevant data); 1 — explains why in context of a large dataset

✗ A weak answer

er response "(a) SELECT * WHERE qty=3 (b) It's faster." Examiner: wrong operator (= not >), missing FROM and ORDER BY, selects all columns not the two asked. (b) too vague — no reason. 1.

✓ A strong answer

er response "(a) SELECT item, qty FROM orders WHERE qty > 3 ORDER BY qty DESC; (b) The query returns only the matching rows and columns automatically, so on a large dataset it's far faster and less error-prone than visually scanning thousands of rows by hand." Examiner: correct clauses and a justified advantage. 4.

Q4. Practice question

(5) (a) 1 — verification (b) 1 — any plausible cause (mistyped value, bad import, manual edit)
(c) 1 — names test-table columns; 1 — expected derived from source first ; 1 — compares expected vs actual to locate the error

✗ A weak answer

er response "(a) Validation. (b) The chart was wrong. (c) She tests it to see if it works."

Examiner: (a) should be verification, not validation. (b) restates the question. (c) no mention of test-table structure. 1.

✓ A strong answer

er response "(a) Verification. (b) A value was mistyped or re-entered when the data was imported into the charting tool. (c) Mia builds a test table with columns *test #*, *description*, *test data*, *expected result*, *actual result*. She lists each item, records the expected bar value calculated from the spreadsheet, then compares it to the chart's actual value — any row where $\text{expected} \neq \text{actual}$ flags the faulty bar." Examiner: precise term, plausible cause, and a structured test-table method. 5.